

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3
**Разработка одностраничного веб-приложения (SPA) с
использованием фреймворка Vue.JS**

Выполнила:

Чунихина Валерия

Группа J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.JS.

Каким требованиям ваше приложение должно соответствовать:

- Должен быть подключён роутер
- Должна быть реализована работа с внешним API (желательно посредством axios)
- Разумное деление на компоненты (продемонстрируйте понимание компонентного подхода)
- Использование composables для выделения повторяющегося функционала в отдельные файлы

Ход работы

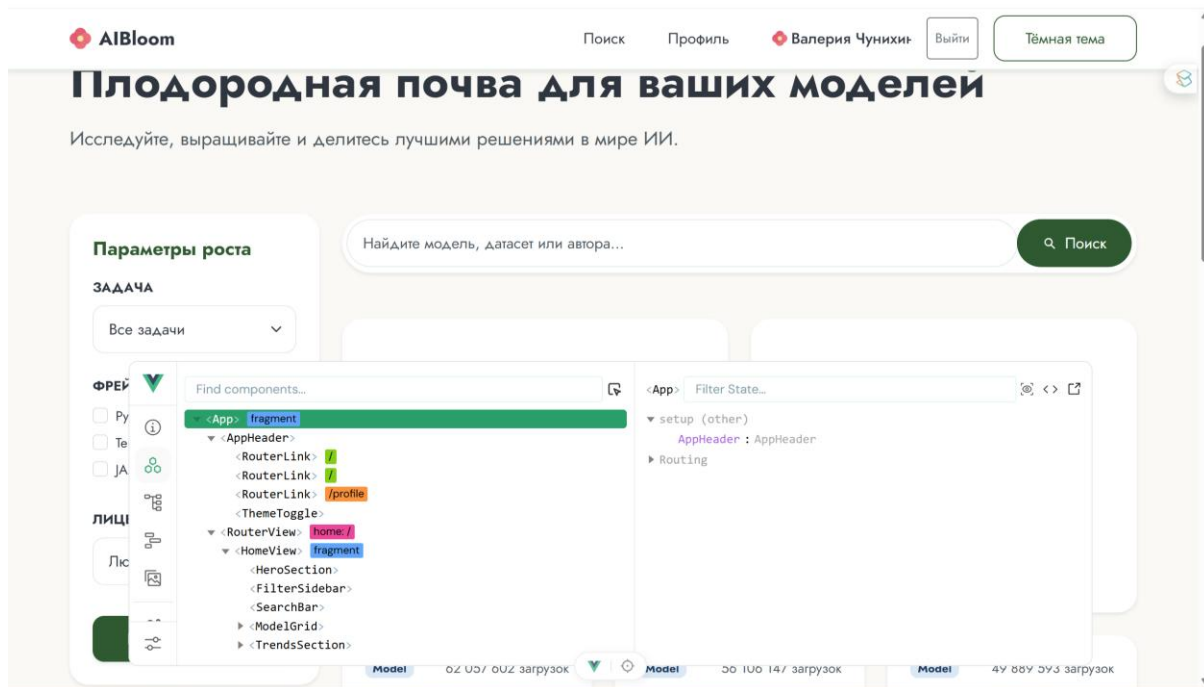
1. Инициализация проекта

Создаю проект с помощью Vite + Vue 3 (npm create vue@latest lab3)
При создании были выбраны опции: Vue Router, без Pinia (состояние управляется через composables). В качестве CSS-фреймворка подключён Bootstrap 5 через.

2. Настройка маршрутизации

В файле src/router/index.js описаны пять маршрутов:

Путь	Компонент	Описание
/	HomeView	Главная страница с поиском и лентой моделей
/details/:slug	DetailsView	Карточка конкретной модели (локальная или с HF API)
/login	LoginView	Форма входа в аккаунт
/register	RegisterView	Форма регистрации нового пользователя
/profile	ProfileView	Личный кабинет авторизованного пользователя



3. Работа с внешним API через Axios

Файл `src/api/http.js` создаёт настроенный axios-инстанс с базовым URL Hugging Face API (<https://huggingface.co/api>).

Файл `src/api/models.js` инкапсулирует три метода:

- `fetchModels(query, filter)` - поиск моделей и датасетов по ключевому слову с фильтрацией
- `fetchTrendingModels()` - получение популярных моделей без поискового запроса
- `fetchModelById(modelId)` - получение данных конкретной модели для страницы деталей

Для авторизации (вход/регистрация) используется отдельный axios-запрос к локальному файлу `/db.json`.

4. Создание компонентной структуры

Приложение разделено на компоненты согласно принципу единственной ответственности:

- `AppHeader.vue` шапка с навигацией, отображением текущего пользователя и кнопкой выхода
- `ThemeToggle.vue` переключатель светлой / тёмной темы
- `home/HeroSection.vue` приветственный баннер главной страницы
- `home/SearchBar.vue` строка поиска с обработкой ввода

- home/FilterSidebar.vue боковая панель фильтров по типу и библиотеке
- home/ModelCard.vue карточка одной модели / датасета с поддержкой вариантов отображения
- home/ModelGrid.vue сетка карточек с логикой отображения результатов и состояния загрузки
- home/TrendsSection.vue блок популярных моделей

5. Функции composable

Повторяющийся и сложный функционал выделен в пять composable-файлов в src/composables/:

- useAuth.js логин, регистрация, выход; реактивный currentUser на уровне модуля обеспечивает глобальное состояние сессии без Pinia
- useCatalog.js состояние поиска, запросы к Hugging Face API, фильтрация и разделение контента
- useDetails.js загрузка карточки модели: сначала проверяется локальный источник (appData), затем HF API по route.query.model
- useProfile.js данные профиля, управление личным инвентарём (модели и датасеты), подписки
- useTheme.js переключение и сохранение темы через localStorage с инициализацией при монтировании

Вывод

В ходе лабораторной работы было успешно мигрировано приложение AIBloom на фреймворк Vue.JS 3. Все четыре обязательных требования выполнены: подключён Vue Router с пятью маршрутами, реализована работа с внешним Hugging Face API через axios, приложение разбито на тематические компоненты с понятной ответственностью каждого, а повторяющийся функционал вынесен в пять composable-функций.

Работа с Vue 3 позволила убедиться в преимуществах реактивной системы: состояние сессии пользователя (currentUser) реализовано через модульный ref без сторонних стейт-менеджеров, а watchEffect в useDetails автоматически перезапрашивает данные при изменении маршрута. Composable-подход оказался удобным, функции переиспользуемы, легко тестируются и не требуют контекста компонента.